

02 · PRD

Milestone. CP-M2 · Sep 27 **Status.** Draft **Author.** Tommy Brown **Working title.** Studio OS (placeholder, rename later, per Decision #1, does not block build). **Input.** [01-concept-brief.md](#), [backlog.md](#), [decisions/0002-cp-m2-founder-decisions.md](#), [research/](#)

Overview

Studio OS is a web application and central portal that runs the business side of an independent dog trainer's or groomer's day. Clients book online through a page the owner can embed on their own website or share as a link, availability comes from the owner's own calendar, automated texts cut no-shows, and past clients are nudged to rebook on their grooming cycle or through a training package. It is sold to the owner-operator on a flat monthly subscription and is designed to be acquired at scale through Meta ads, which are a later upsell.

The wedge is that most of these owners have almost no digital infrastructure. They run on back-and-forth texts, phone tag, and a paper book. Studio OS removes that communication work instead of making them learn yet another app.

Every requirement traces to a backlog story (S1–S10 in [backlog.md](#)); founder decisions are recorded in [0002-cp-m2-founder-decisions.md](#).

Design principle (platform-wide)

The platform provides the infrastructure; the owner configures the business logic. Studio OS supplies the foundation (scheduling, reminders, payments, records) and exposes every business-specific behavior, time zone, cancellation rules, deposit/refund policy, rebooking cadence, buffer time, as an owner-configurable setting. The platform then operates according to whatever the owner set; it never forces one fixed way of running a business across every owner. (Decision #15.) This is the reason owners can adopt it without changing how they already work.

Problem definition

What is the problem? Independent trainers and groomers have no digital infrastructure to scale. Booking, rescheduling, reminders, and rebooking all happen through manual back-and-forth: a text here, a missed call, a note in a book. It's slow, it drops leads, and it caps how big the business can get because the owner is the bottleneck for every message.

Who is facing the problem? Solo (or 2–4 person) dog trainers and groomers with little to no software. Many are older, have run the business for years on a notebook and a phone, know they're leaving money on the table, and don't know how to take the business to the next level or where to start with tech.

Business value unlocked - Decreased time spent on admin and messaging. - Automatic booking (self-serve on the owner's site or link, against their real calendar). - Automatic retainers/rebooking that bring clients back on cycle. - Better client experience and a more professional image. - Reduced "customer support" load (fewer calls/texts the owner personally answers). - More data: one central portal with clients, dogs, revenue, and history.

How will the target users benefit if the problem is solved? - They stop losing money to no-shows and unreturned messages; the booking page works 24/7 while they're with a dog. - Their calendar fills itself: freed slots get filled from a waitlist, and past clients get auto-nudged to rebook on their cadence. - They look bigger and more professional than competitors still working out of a notebook. - They finally see their business in one place and can grow without hiring a front desk. - They keep running their business their way, because every policy is their setting, not ours.

Why is it urgent now? Owners are more open to tech than ever, especially with AI, but they don't know where to start. Whoever gives them a dead-simple, done-for-you on-ramp now wins the relationship before the incumbents reach down-market.

Go-to-market: the offer

The Meta ad hook, aimed straight at trainers/groomers:

"What if I could increase your bookings **and** hand you a portal that runs your scheduling, reminders, and rebooking for you, all in one?"

Meta ads are a planned upsell, not part of the current build. A Meta Pixel is embedded by default on every owner's public booking widget now (the widget is effectively their landing page); the Conversions API event taxonomy and attribution logic are deferred (Decision #12).

Goals

Primary goal - Remove the manual back-and-forth: let clients book themselves against the owner's calendar, cut no-shows with automatic texts, and auto-rebook, so an owner can grow without a front desk.

Secondary goals - Same-day activation for non-technical owners (done-for-you setup, import contacts, connect a calendar or create one). - Let every owner run their business their way: deposits, refunds, cancellation, and cadence are their configurable settings, not our fixed rules (Decision #10, #15). - Give trainers a reason to stay (visible session progress) and groomers a reason to stay (auto-rebooking that fills the calendar).

Last goal (lowest priority) - Rewards/discounts (a simple loyalty perk) once the core loop is proven.

Non-goals (explicit) - No facility/enterprise features (boarding, daycare, kennel, multi-location chains). - No consumer marketplace, and no taking a % of the owner's revenue (flat subscription only). - No native mobile app this semester (mobile web + SMS only). - No AI receptionist / automated Instagram-DM booking (not reliably scalable; Meta gates the messaging API). Auto-SMS covers the automated-communication need. - No Meta CAPI/attribution build yet (Pixel only for now, Decision #12). - No veterinary/medical records or health-regulated data.

User personas

Sandy, 58, groomer of 25 years (small shop). Runs everything from a spiral notebook and her cell. Full grooms every 4–8 weeks. Pain: misses calls while grooming, forgets to remind clients, never nudges rebooks, "not a computer person."

Dana, 34, independent dog trainer. Private + in-home lessons and board-and-train, booked over calls/DMs, progress in a spreadsheet. Pain: evenings lost to admin; clients drop mid-package; no package or progress tooling.

Marcus, 41, mobile groomer. Books by phone/text between stops. Pain: no-shows waste a slot and a drive; MoeGo is too complex and charges per van with metered texts.

Secondary actor — the pet owner (end client). Books online, gets reminders, leaves reviews. Not the buyer, but their experience drives the owner's willingness to pay.

Core features

1. **Self-serve booking (S1)** — services by type (grooming/training) and size/duration, each with price and optional deposit; instant confirmations. A hard rule: bookings cannot be made less than 24 hours in advance; the flow blocks near-term self-serve bookings (Decision #7).
2. **Availability from the owner's calendar (S1, Decision #3)** — availability is derived from the owner's own Google or Apple Calendar (busy blocks → open slots). Owners without a

calendar can have one created inside the setup flow. Calendar sync is a first-class MVP integration, not optional.

3. **Multi-staff scheduling + buffers (Decision #4)** — multiple staff, each with their own hours; book different staff at different times, or the same slot across different staff (multi-provider). Buffer time between bookings is an owner setting.
4. **Two-way calendar sync + conflict resolution (Decision #5)** — each staff member's bookings write to their own connected calendar. On a conflict between an external event and a Studio-side booking, a popup lets the owner/staff override or dismiss; override defaults to favoring the Studio-side booking.
5. **Embeddable booking, everywhere (S1)** — a booking widget the owner drops onto their own site with one snippet, plus a shareable link for Instagram bio/Google/texts, and a hosted page if they have no website. A Meta Pixel is embedded on this widget by default.
6. **Automated SMS reminders (S2)** — 24h reminder with a cancel/reschedule link; cancel reopens the slot and notifies the owner; every send + outcome logged. Simple keyword replies (reply C to confirm, cancel, STOP). Unrecognized replies get a single auto fallback pointing to a human contact path (owner's link/number), logged like any other reply (Decision #8, provisional).
7. **Owner dashboard + "Revenue Recovered" (S3)** — today/this-week view with empty and error states, plus a running tally of money saved from prevented no-shows, filled waitlist slots, and auto-rebookings.
8. **SMS consent capture (S4)** — explicit, timestamped opt-in; no consent, no texts; honors STOP.
9. **Auto-rebooking on cadence (S5)** — per-service rebook cycle; one nudge with a link; suppressed if a future appointment exists (exact suppression scope is an open CP-M3 question, see below).
10. **Automated review requests (S6)** — one post-visit review request by SMS with a direct link. The platform marks the request **sent**; it does not verify a review was actually posted (Decision #13).
11. **Client/dog records + session notes (S7)** — notes/homework per session, viewable by the client via a private link (no app), the training wedge.
12. **Waitlist auto-fill (S8)** — freed slots offered to the next waitlisted client by SMS with a claim window. (Data-model shape is an open CP-M3 decision, see below.)
13. **Owner-configured policies (Decision #10, differentiator)** — deposit, refund, bundle, and cancellation rules are the owner's settings; the platform enforces whatever they configure and imposes no fixed policy.
14. **Payments: packages & deposits (S10)** — deposits and multi-session packages through the owner's own processor (**Stripe or Square**, Decision #9).
15. **Rewards/discounts (last, lowest priority)** — a simple loyalty perk once the core loop is proven.

What makes it hard to say no to

- **Run your business your way.** Every policy, deposit, refund, cancellation, cadence, is the owner's setting. Studio OS gives them the infrastructure without forcing them to change how their business already runs (Decision #10, #15). This is the core positioning.
- **Done-for-you setup for non-techy owners.** Import contacts, connect (or create) a calendar, and the portal + booking page are built in minutes.
- **It works over text.** Reminders and rebooking run on SMS; a 58-year-old groomer adopts it in a day.
- **ROI you can see.** The "Revenue Recovered" number turns the subscription into obvious profit.
- **Embeddable anywhere.** One snippet on their site, one link in their bio.

UI / UX

- **Mobile-first, dead simple.** Big touch targets, minimal steps, plain language, no jargon. Calm, premium, Apple-inspired.
- **Owner app:** home dashboard (today's schedule, alerts, Revenue Recovered), tap-to-add booking, client/dog list, settings (hours, buffers, policies, cadence, time zone), calendar-conflict popups.
- **Client booking page/widget:** branded to the owner, 3 taps to book (service → time → details), availability from the owner's calendar, clear consent + deposit steps, confirmation; embeddable or hosted.
- **Every screen has an empty state and an error state** (per `CLAUDE.md`).
- **Accessibility:** large fonts, high contrast, SMS fallbacks.

Payment processing

- **Processor is the owner's choice: Stripe or Square** (Decision #9). Built as a processor abstraction in the service layer, not a Stripe-only integration.
- Each owner connects their own processor account; money lands with them, not the platform.
- Deposits required at booking on selected services; packages sold as prepaid bundles. Deposit, refund, and cancellation behavior follow the owner's configured policy (Decision #10).
- Subscriptions (Starter/Pro/Team) billed to the owner; failed-payment retries and dunning, then a defined offboarding on repeated failure (see Account lifecycle).
- No percentage cut of the owner's revenue, ever.

Account lifecycle & offboarding (Decision #11)

If an owner's subscription fails repeatedly and the account is shut down, the shutdown flow must leave the platform with no liability for the owner's data, active bookings, or client obligations:

- A defined offboarding state with a data-export window for the owner.
- The client-facing booking page/widget is taken down cleanly (no dead booking links).
- No orphaned scheduled jobs, all pending reminders/nudges for the shuttered account are cancelled.

User flow

Owner side 1. Clicks a Meta ad, lands on the offer page, starts a trial or books a demo. 2. Done-for-you setup: imports contacts; connects Google/Apple Calendar (or has one created); sets services, hours, buffers, and policies. 3. Connects a processor (Stripe or Square) and registers under the platform's SMS sender. 4. Embeds the booking widget on their site and drops the link in their Instagram bio. 5. Runs the day from the dashboard; resolves any calendar-conflict popups; watches Revenue Recovered climb.

Customer (pet owner) side 1. Finds the business (ad, the owner's website, or IG bio link). 2. Books through the embedded widget or link; picks a service and an open slot (≥ 24 h out). 3. Enters details + dog, gives SMS consent, pays a deposit if the owner requires one. 4. Gets a confirmation, then a 24h reminder (cancel reopens the slot). 5. Visit happens; for training, views session notes/homework via a private link. 6. Gets a review request; later gets one rebooking nudge on cycle and books again.

Database

Postgres (via Supabase) with per-account row-level security (RLS policy design is a CP-M3 task).
Core entities:

- **Account** (owner/business) — business_name, owner_name, email, phone, plan (starter/pro/team), **payment_processor (stripe/square)**, **timezone**, **default policy fields** (deposit rule, refund rule, cancellation rule, default rebook cadence, default buffer_minutes).
- **Staff** — belongs to Account; name, role (owner/staff), **working_hours**, **buffer_minutes** (override). For multi-provider scheduling.
- **CalendarConnection** — belongs to Staff; provider (google/apple), external_calendar_id, auth token ref, sync_state. Read for availability, write for bookings (Decision #3, #5).
- **Service** — belongs to Account; name, type (grooming/training), size_tier, duration_min, price, requires_deposit, deposit_amount, rebook_cycle_days, **policy overrides**

(deposit/cancellation), active.

- **Client** — belongs to Account; name, phone, email, sms_consent, sms_consent_at.
- **Dog** — belongs to Client; name, breed, size, notes.
- **Booking** — belongs to Account, Client, Dog, Service (and a specific Staff); start_time, end_time, status (pending/confirmed/completed/cancelled/no_show), source (organic/meta/referral), external_event_id (the written calendar event).
- **Lead** — belongs to Account; name, phone, source, status (new/contacted/booked/lost).
- **Reminder** — belongs to Booking; channel (sms), scheduled_for, sent_at, status, outcome (confirmed/cancelled/none/fallback).
- **SessionNote** — belongs to Booking + Dog; notes, homework, share_token.
- **ReviewRequest** — belongs to Booking; sent_at, review_link, status (**pending/sent only** — no verification, Decision #13).
- **Package** — belongs to Client + Service; sessions_total, sessions_used.
- **Payment** — belongs to Account + Client, optional Booking or Package; amount, type (deposit/package/full/subscription), **processor (stripe/square)**, **processor_payment_id**, status.
- **Reward** (last-priority) — belongs to Account + Client; type, threshold, status.
- **ConversionEvent** (deferred with Meta CAPI) — belongs to Booking; source, utm, meta_event_id, fired_at. Pixel fires client-side now; server-side CAPI events are later (Decision #12).

Open modeling questions for CP-M3 (Decision #14) - Waitlist: model a waitlisted client as a Booking with a `waitlisted` status, or as its own `WaitlistEntry` entity? Decides how claim windows and expiration work. - **Rebook suppression scope:** suppress on any future appointment for the client, or only the same client + same service? Needs a founder call (grooming vs training behave differently). - **Lead conversion events:** should a Lead that never books still fire a top-of-funnel Meta signal? Revisit when CAPI work starts.

Key relationships - Account has many Staff, Services, Clients, Bookings, Leads. Each Staff has one CalendarConnection. - A Client has many Dogs and Bookings. A Booking belongs to a specific Staff and has its Reminder(s), an optional SessionNote, one ReviewRequest, and an optional ConversionEvent. - A Payment belongs to an Account + Client and optionally a Booking or Package.

Backend

- Supabase (Postgres, Auth, Storage) with per-account row-level security (policies designed at CP-M3).
- **Service-layer abstractions** so vendors are swappable: a `PaymentProcessor` interface (Stripe, Square) and a `CalendarProvider` interface (Google, Apple), plus the SMS provider.

- **Config-driven feature gating (Decision #2):** plan → feature mapping lives in configuration, not hardcoded, so Starter/Pro/Team contents can change (backlog B1) without a schema migration.
- **Job queue** for scheduled, idempotent work (reminders, rebooking nudges, review requests, waitlist offers). Engine choice, n8n vs Supabase scheduled functions, is a CP-M3 decision (Decision #14).
- Webhooks: inbound SMS (keyword replies + fallback), processor payment status, calendar change notifications.
- On account shutdown, a cleanup job cancels all pending scheduled work for that account (Decision #11).
- Logging + alerting on delivery failures and job errors.

Frontend

- Next.js / React / Tailwind, mobile-first, server-rendered.
- Owner app (dashboard, bookings, clients/dogs, settings) and a branded public booking page, plus an embeddable booking widget (a small script the owner pastes into their site) with the Meta Pixel.
- Component library with built-in empty/error/loading states; calendar-conflict popup component.
- Real-time dashboard updates via Supabase subscriptions.

API integration points (first-class)

- **Calendar (Google Calendar, Apple Calendar)** — read availability (busy blocks → open slots) and write staff bookings; core MVP integration (Decision #3, #5).
- **Twilio** — reminders, nudges, review requests, keyword replies + fallback; A2P 10DLC (see SMS strategy).
- **Stripe Connect / Square** — deposits, packages, subscriptions via a processor abstraction; webhooks for status (Decision #9).
- **Meta Pixel** — embedded on the booking widget now; **Meta Conversions API deferred** (Decision #12).

SMS number strategy & A2P 10DLC (needs a CP-M3 decision)

Founder's stated plan (Decision #6): the founder owns A2P 10DLC registration and runs a single shared number across all owner accounts; overage beyond the fair-use cap bills automatically.

Risk / open decision. A single shared number across many unrelated businesses is a known A2P 10DLC failure mode: carriers/Twilio expect one brand + one campaign per distinct business use case, and mixing unrelated businesses behind one sender gets throttled, delisted, or rejected in vetting. It also pools deliverability reputation, one bad actor degrades everyone's delivery.

Recommended path (to decide at CP-M3): Twilio subaccounts under a shared brand registration with per-owner (or pooled, smaller-batch) campaigns, likely via a reseller/ISV program built for multi-tenant SMS. Resolve before locking the Twilio design, retrofitting number architecture after owners are live is disruptive. The keyword-reply fallback (Decision #8) is part of this same decision.

Technical requirements

1. Booking shows real availability from the owner's calendar and prevents double-booking. Why? A confirmed-but-taken slot destroys trust. (S1, #3) — Frontend, Backend, Calendar
2. The booking flow blocks any self-serve booking less than 24h out. Why? Hard business rule. (#7) — Frontend, Backend
3. The booking widget embeds on any owner site with one snippet and carries the Meta Pixel. Why? Owners use different site builders; the widget is the landing page. (S1, #12) — Frontend
4. Multi-provider scheduling with per-staff hours and configurable buffers. Why? Real shops run several providers in parallel. (#4) — Backend, Frontend
5. Each staff booking writes to that staff's connected calendar; conflicts raise an override/dismiss popup (default favors Studio-side). Why? Keep one source of truth without silent double-books. (#5) — Calendar, Frontend
6. Payments run through a processor abstraction (Stripe or Square) selected per owner; Payment stores the processor. Why? Owners already use one or the other. (#9) — Backend
7. Deposit/refund/cancellation/cadence/buffer/timezone are owner-configured settings the platform enforces; no hardcoded policy. Why? Core differentiator + principle. (#10, #15) — Backend, Frontend
8. Feature gating is config-driven (plan → features), changeable without a schema migration. Why? Tiers will shift (B1). (#2) — Backend
9. Reminders/nudges run as idempotent scheduled jobs; unrecognized SMS replies get a logged fallback. Why? Fire once, on time; nothing silently disappears. (S2, S5, #8) — Job queue, Twilio
10. SMS consent stored with timestamp + wording and enforced before any send; honor STOP. Why? Legal (TCPA / A2P 10DLC). (S4) — Backend, Compliance
11. Account shutdown enters a clean offboarding state: export window, booking page down, all pending jobs cancelled. Why? No platform liability, no orphaned reminders. (#11) —

Backend

12. Every view has explicit empty and error states. Why? Blank/broken reads as "it failed." (S3)
— Frontend

Non-functional requirements

- **Security & tenancy:** auth/authorization, per-account Postgres RLS (policies drafted at CP-M3), secret storage, input validation, expiring tokenized links.
- **Compliance:** SMS consent + STOP handling; A2P 10DLC (number/brand/campaign strategy per above).
- **Performance:** booking page/widget p95 < 2s on mobile; reminder dispatch within 1 min of schedule.
- **Reliability:** high SMS delivery success; jobs retry with alerting; no orphaned jobs after shutdown.
- **Environments:** sandbox/staging for Twilio, Stripe/Square, and calendar so integration testing never hits live owner accounts or live SMS (Decision #14).

Pricing (business model)

Flat, feature-gated tiers; no per-seat/per-van; texts included (fair-use cap, overage bills automatically). Owner connects their own processor. **Tier contents are provisional and gating is config-driven; expect boundaries to shift (backlog B1, Decision #2).**

Plan	\$/mo flat	Adds (provisional)
Starter	29	booking, calendar sync, reminders, reviews, fair-use texts
Pro (hero)	49	+ rebooking automation, packages, waitlist, session notes, multi-staff
Team	99	+ more staff/calendars, rewards/discounts, no per-seat trap

What we'll watch (plain terms)

- Are no-shows going down for reminded clients?
- Are clients booking themselves instead of the owner doing it by hand?
- Are past clients coming back (rebooking)?
- Does a Meta ad turn into a paying owner for a reasonable cost (once ads are on)?

Open questions carried into CP-M3

- SMS number / A2P 10DLC strategy (shared number vs subaccounts + per-owner campaigns) and the keyword-reply fallback (Decisions #6, #8).
- Waitlist entity modeling; rebook-suppression scope; lead conversion events (Decision #14).
- Job queue engine (n8n vs Supabase scheduled functions) and RLS policy design (Decision #14).
- Product name (Decision #1); final tier split (Decision #2 / B1).

Dependent stakeholders

- Google/Apple (calendar API access), Twilio (A2P 10DLC brand/campaign approval), Stripe & Square (processor onboarding), Meta (Pixel now; business verification + ad account later), and early pilot pros for validation.

Next

CP-M3 · Architecture · Oct 11 — [docs/03-architecture.md](#)